Project 5 — ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-Database-And-Docker

Title: ContentForge Pro Production Ready Content Pipeline With API Database And Docker

Type: Multi-agent service (Project 4's 5 agents, behind an API) · **Stack:** Python, FastAPI, SQLite, Docker, pytest, LangGraph · **Level:** Upper-Intermediate

Tagline: ContentForge (Project 4), served as a real API — every run saved, packaged in a container, rate-limited, and guarded by tests that catch a bad change before it ships.

> This is the **reading guide** — the story, the folder structure and why each part exists, setup, the 4 build steps, and which document to read for each idea. The other file, README.md, is the **repo README** — you copy it into your repo folder at the end and push it to GitHub. You write every folder, file and line of code yourself; the only thing already created for you is the empty repo folder `ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-Database-And-Docker/`.

Charter (what this project is)

Project 4, made production-ready: served over FastAPI, every run saved to SQLite, packaged with Docker, rate-limited, and covered by tests at every layer — API tests with the team faked, and an eval suite that scores the real articles. **No new agents are built here** — Project 4's code is copied in unchanged. This project is about running those agents safely for other people.

Jump to: Read first · Project structure · Setup · Step 1 · Step 2 · Step 3 · Step 4 · Run, test, configure · Checklist · Publish

The Story — what you're actually building

Picture ContentForge as a very capable employee who only works if you sit at their desk and run their code by hand. They do good work, but nobody else can use them: there's no phone number to call, no record of what they did, and no way to trust them with many callers at once.

Project 5 turns that employee into a service. Step 1 gives it a phone number — an HTTP address anyone can call with a topic. Step 2 gives it a filing cabinet — every run saved to a database, findable by ID. Step 3 puts it in a shipping container (Docker), so it runs the same on any machine,

and adds a bouncer at the door (a rate limit). Step 4 adds a quality inspector — a test suite that notices when a change makes the articles worse.

None of these steps touch the five agents. The difference is the safety net around them: "a cool thing on my laptop" versus "a service I could hand to someone else".

What you're building, in one line: Project 4's 5-agent graph behind a FastAPI service, backed by SQLite, packaged with Docker, and guarded by API tests and an eval suite.

Why this needs to exist: a multi-agent system that only runs from a terminal can't be called by a web page, another service or a teammate — and without saved runs and tests, nobody can tell what went wrong, or whether a change made it worse.

When you'd reach for this at a real job: whenever a working prototype must become a service other people or systems call — this is what "shipping" an agent looks like.

How it works, mechanically: a `POST /run-task` route calls **one** function, `run_pipeline(topic)`, which runs Project 4's graph. The route saves a `"started"` row first and updates it after, tags every log line with a request ID, and turns a crash into a plain 500. Docker packages it with the key passed in only at run time. pytest checks the API with the team faked; an eval suite scores the real articles.

Why not keep running it as a script? Then only you can use it, nothing is recorded, and "it works on my machine" is the only promise. **Why not check changes by eye?** A change that makes articles a bit worse passes a quick look — an automated suite with a pass bar doesn't.

Where This Fits in the 5-Project Arc

```
Project 1               → 2 agents, Python router, raw SDK
Project 2               → 2 agents, hand-built agent loop, tools
Project 3               → 3 agents, LangGraph, RAG + human approval
Project 4               → 5 agents, one Supervisor directs the rest
Project 5 (you are here) → Project 4, made production-ready
                           (API + database + Docker + tests)
```

New here: nothing about the agents — everything **around** them. Project 4's code is the starting point, copied in unchanged.

What to Read First (docs this project uses)

You need to understand…	Read this	Used in
A route, `response_model`, `TestClient`, `APIRouter`	Doc12 — health route	Step 1
400 vs. 500, `logger.exception`, request IDs	Doc12 — chat route	Steps 1, 2
Saving runs, `GET /runs/{id}` with a 404	Doc12 — SQLite persistence	Step 2
`INSERT` → `lastrowid` → `UPDATE`, `with conn:`	Doc12 — validation 422	Step 2
`Dockerfile`, `.dockerignore`, the startup check	Doc12 — dead connection and container	Step 3
The whole API and the rate limiter, as a practice run	Doc12 Build Task	Steps 1-3
Test layers, faking the model	Doc13 — testing layers	Steps 1, 4
`monkeypatch`	Doc13 — test a tool without the LLM	Step 1
pytest `assert` tests	Doc13 — a normal, exact test	Step 4
Property checks, not exact wording	Doc13 — fix a flaky test	Step 4
A judge with written rules	Doc13 — score an answer	Step 4
Proving the suite catches a regression	Doc13 — regression catch	Step 4
The whole suite, as a practice run	Doc13 Build Task	Step 4
The team you're wrapping	Project 4	Setup

Reused code: all of Project 4, unchanged. `db/database.py`, the rate limiter, the request-ID middleware, the `Dockerfile` and `.dockerignore` are Doc12's Build Task (Intermediate); `judge.py`, `run_eval.py` and `regression_check.py` are Doc13's Build Task (Intermediate), pointed at `run_pipeline()`.

Project Structure (the professional layout)

Same rule as before: only folders with a real job. New here: `api/` (the web layer), `db/` (the run history), `evals/` (article quality), `pipeline.py` (the one door into the team), `tests/api/`, and the two Docker files at the root.

Names: the repo folder vs. the Python package

- **Repo folder:** `ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-Database-And-Docker/` — already created, and the exact GitHub repo name.
- **Python package:** still `contentforge` (inside `src/`). **Why keep the name:** Project 4's files come in unchanged — every `from contentforge...` import keeps working, and the new code sits beside it (`from contentforge.api.routes import router`).
- **Files:** `snake_case.py`; test files start with `test_`; the app uvicorn starts is `api/main.py` — the terminal app stays `main.py`.

The final layout (after Step 4)

```
<repo folder>/   (ContentForge-Pro-...-API-Database-And-Docker/)
├── README.md               the repo's front page (copied in at the end)
├── LICENSE                 lets others reuse your code (MIT)
├── .gitignore              keeps .env, .venv/, runs.db out of git
├── .env.example            the setting names, no real values
├── .env                    your real key — never committed
├── pyproject.toml          makes src/contentforge an installable package
├── requirements.txt        the libraries, for pip install -r
├── Dockerfile              builds the API's container image
├── .dockerignore           keeps .env and runs.db out of the image
├── src/
│   └── contentforge/
│       ├── __init__.py
│       ├── main.py             asks for a topic, prints the report
│       ├── config.py           Config, load_config()        (Doc01)
│       ├── exceptions.py       MissingConfigError           (Doc01)
│       ├── schemas.py          ReviewVerdict (score, feedback)
│       ├── state.py            TeamState, start_state()
│       ├── graph.py            build_graph(): 5 nodes, 1 edge
│       ├── pipeline.py         run_pipeline(): the one door in
│       ├── models/
│       │   ├── __init__.py
│       │   └── llm.py          get_model()              (Project 3)
│       ├── agents/
│       │   ├── __init__.py
│       │   ├── supervisor.py      decide_next(), supervisor()
│       │   ├── research_agent.py  run_research(), research_node
│       │   ├── analysis_agent.py  run_analysis(), analysis_node
│       │   ├── writer_agent.py    run_writer(), writer_node
│       │   └── reviewer_agent.py  run_reviewer(), reviewer_node
│       ├── tools/
│       │   ├── __init__.py
│       │   └── lookup.py       lookup_facts (@tool)
│       ├── prompts/
│       │   ├── research.txt
```

```
│       │   ├── analysis.txt
│       │   ├── writer.txt
│       │   └── reviewer.txt
│       ├── utils/
│       │   ├── __init__.py
│       │   ├── logger.py       get_logger()                (Doc01)
│       │   └── prompts.py      load_prompt(name)        (Project 1)
│       ├── api/
│       │   ├── __init__.py
│       │   ├── main.py         the app uvicorn starts       (Doc12)
│       │   ├── routes.py       /health, /run-task, /runs/{run_id}
│       │   └── schemas.py      TaskRequest, TaskResponse    (Doc12)
│       ├── db/
│       │   ├── __init__.py
│       │   ├── database.py     save and fetch runs (SQLite) (Doc12)
│       │   └── models.py       Run: one saved run's shape   (Doc12)
│       └── evals/
│           ├── __init__.py
│           ├── tasks.py        10 fixed topics and checks   (Doc13)
│           ├── judge.py        score_with_judge()           (Doc13)
│           ├── run_eval.py     runs the suite, saves report.json
│           └── regression_check.py  breaks a prompt, checks the drop
├── tests/
│   ├── unit/
│   │   ├── test_lookup.py      no API key needed
│   │   └── test_supervisor.py  no API key needed
│   ├── api/
│   │   └── test_routes.py      the API, with the team faked
│   └── integration/
│       ├── test_live_agents.py real API calls
│       └── test_judge.py       real API calls
└── data/
    └── sample/
        └── facts.json          the facts Research can look up
```

Files the app makes as it runs — `runs.db` (Step 2) and `report.json` (Step 4) — are in `.gitignore` and `.dockerignore`.

Why each folder exists

- **`src/contentforge/`** — Project 4's package, unchanged, plus `pipeline.py`: the one function the API and the evals call to run the team. **Why:** a test swaps one function for a fake, and nothing outside the package touches the graph's state.
- **`api/`** — the web layer: the app, the routes, and the request/reply shapes. **Why:** the HTTP rules (status codes, rate limit, request IDs) live apart from the agents.

- **`db/`** — every SQL statement behind small functions, plus a saved run's shape. **Why:** the routes read like a story, and the SQL is in one place.
- **`evals/`** — the fixed topics, the judge, the runner and the regression check. **Why:** article quality is measured with the real model, on purpose — not on every save.
- **`agents/`**, **`tools/`**, **`prompts/`**, **`models/`**, **`utils/`** — Project 4's, unchanged.
- **`tests/unit/`** — the lookup tool and the routing rules. No key, no cost.
- **`tests/api/`** — every route and status code, with the team faked by `monkeypatch`. No model calls, no cost.
- **`tests/integration/`** — the real team and the real judge. Costs money, so it's run on purpose.
- **`data/sample/`** — Project 4's fact store, unchanged.

Why each root file exists

- `README.md` — the repo's front page on GitHub.
- `LICENSE` — lets others legally reuse your code (GitHub can add MIT).
- `.gitignore` — keeps secrets, the venv, `runs.db` and `report.json` out of git.
- `.env.example` — which settings to create, with no real values.
- `pyproject.toml` — the package's name, libraries and the `contentforge` command.
- `requirements.txt` — the libraries, plus the test tools, for `pip install -r`.
- `Dockerfile` — the recipe that builds the API's image.
- `.dockerignore` — keeps `.env` and local data out of that image.

Why each file exists (what and why)

Every file you'll write, in one place — the same What/Why list the documents give after each tree. The steps below build these files one at a time; come back here whenever you forget why a file exists.

- `__init__.py` (every code folder)
  - **What:** An empty file. You create one in every folder that holds Python code — nothing goes inside it.
  - **Why:** Python only treats a folder as a package when this file is there. Without it, an import like `from <package>.<folder> import ...` fails with `ModuleNotFoundError`.
- `main.py`
  - **What:** `main()` loads the config, builds the graph, reads a topic, runs `graph.invoke(start_state(topic))`, and passes the result to `print_report(result)`. `print_report()` (written in Step 2, extended in Step 5) prints the routing log and then the article, the "couldn't agree" report, or "nothing is known".
  - **Why:** One starting point that reads the topic and prints the result. The team and its routing live in other files.
- `config.py`

  - **What:** A `Config` dataclass with two fields (`openai_api_key` , `log_level`), a helper `require_env(key)` that raises `MissingConfigError` when a setting is empty, and `load_config()` , which calls `load_dotenv()` , reads both settings and returns a `Config` .
  - **Why:** Every file gets its settings from one place. If the key is missing, the app stops at startup with one clear line — without this file, the missing key would show up later as a confusing API error.
- `exceptions.py`
  - **What:** A few small classes that inherit from `Exception` — at least `MissingConfigError` . Each one is just `class Name(Exception): pass` , with a docstring saying when it's raised.
  - **Why:** Code can catch one specific problem (`except MissingConfigError:`). Without named errors you'd catch every `Exception` , and real bugs would be hidden along with the expected problem.
- `utils/logger.py`
  - **What:** One function, `get_logger(name)` . It gets the logger for that name and, only the first time (`if not logger.handlers:`), adds a console handler with a timestamp format and the level from `LOG_LEVEL` .
  - **Why:** Every file logs in the same format with one line of code. Without the `if not logger.handlers:` guard, every log line would print twice.
- `utils/prompts.py`
  - **What:** One constant, `PROMPTS_DIR` (the prompts folder, from the repo root), and one function, `load_prompt(name)` , which opens `PROMPTS_DIR + "/" + name + ".txt"` and returns its text.
  - **Why:** Prompts live in text files, so you can change how an agent behaves without editing Python — and no agent has to open files itself.
- `schemas.py`
  - **What:** One Pydantic class, `ReviewVerdict` , with `score: int = Field(ge=1, le=10)` and `feedback: str` .
  - **Why:** The Reviewer's score is a checked number the code compares with the pass mark — not a word to search for.
- `state.py`
  - **What:** A `TeamState` TypedDict (`topic` , `research_notes` , `research_done` , `findings` , `draft` , `feedback_history` , `accepted` , `revision_count` , `routing_log`) with a comment naming each field's single writer, and `start_state(topic)` .
  - **Why:** Each field has exactly one writer, so no agent can overwrite another agent's work.
- `graph.py`
  - **What:** One function, `build_graph()` : add the five nodes, add one edge `START → "supervisor"` , and `compile()` .
  - **Why:** Every other move is a `Command` returned by a node, so the Supervisor decides the route while the program runs.
- `models/llm.py`

    - **What:** A `MODEL_NAME` constant and one function, `get_model()`, which returns `ChatOpenAI(model=MODEL_NAME, temperature=0)`.
    - **Why:** Every chain and agent gets the same model and settings from here. To try another model you change one line, not every file.
- `agents/supervisor.py`
    - **What:** `MAX_REVISIONS`, `decide_next(state)` — a chain of `if` checks in a fixed order that returns `(next_step, reason)` — and `supervisor(state)`, which adds `{step, reason}` to the routing log and returns `Command(goto=...)` (or `goto=END`).
    - **Why:** The routing rules are a plain function you can test with a hand-made dict, and every move is logged with its reason.
- `agents/research_agent.py`
    - **What:** `run_research(topic)`: call `lookup_facts.invoke({"topic": topic})`; if it returns `""`, log a warning and return `""`; otherwise run the research prompt chain and return the notes. Plus `research_node(state)`, which calls it and returns `Command(update={notes, research_done: True}, goto="supervisor")`.
    - **Why:** The article starts from looked-up facts, not the model's memory, and an unknown topic stops the team instead of producing invented facts.
- `agents/analysis_agent.py`
    - **What:** `run_analysis(notes)`: run the analysis prompt chain and return the themed findings. Plus `analysis_node(state)`, which returns `Command(update={findings}, goto="supervisor")`.
    - **Why:** The Writer gets a clear structure, and the "keep every fact" rule means nothing is lost on the way.
- `agents/writer_agent.py`
    - **What:** `build_writer_chain()`, `run_writer(findings, feedback="")` (turns empty feedback into "none" and invokes the chain), and `writer_node(state)`, which passes the newest feedback from `feedback_history` and returns the draft.
    - **Why:** It writes the article, and it can improve the draft when the Reviewer asks — using the Reviewer's exact feedback.
- `agents/reviewer_agent.py`
    - **What:** `PASS_SCORE`, `run_reviewer(findings, draft)` (a chain with `with_structured_output(ReviewVerdict)`), and `reviewer_node(state)`: on a pass, set `accepted`; on a fail, add 1 to `revision_count`, add the feedback to the history, and clear the draft if revisions are left.
    - **Why:** A second opinion with written criteria stops weak drafts. Clearing the draft is the signal that sends the work back to the Writer.
- `tools/lookup.py`
    - **What:** `load_facts()` reads `facts.json` with `json.loads()`, and a `@tool` function `lookup_facts(topic)` returns the facts of the first known topic found in the text, one per line — or `""`.

    - **Why:** Research's tool can be tested without any model, so a wrong note can be pinned on the tool or on the agent.
- `prompts/*.txt`
    - **What:** Four text files, one per specialist: research (one note per fact, keep the tag), analysis (themes, keep every fact), writer (only the findings, with a feedback section), reviewer (three written criteria and the scoring rule).
    - **Why:** Each agent's narrow job is written down in its own prompt — the reason for splitting the work in the first place.
- `tests/unit/test_lookup.py`
    - **What:** Two test functions that call `lookup_facts.invoke(...)` with a known and an unknown topic and print the results.
    - **Why:** A wrong topic match shows up here, with no model and no cost.
- `tests/unit/test_supervisor.py`
    - **What:** A `show(label, state)` helper and test functions that build states by hand with `start_state()`, change a few fields, and print what `decide_next()` returns — every step in order and every way to stop.
    - **Why:** Routing mistakes are found without running the team or paying for model calls.
- `tests/integration/test_live_agents.py`
    - **What:** Test functions with the real API: `run_reviewer()` must fail a weak draft, a known topic must go through all four specialists, and an unknown topic must end with "research found no facts".
    - **Why:** It proves the team works from start to finish — and that the Reviewer really rejects bad work.
- `data/sample/facts.json`
    - **What:** A JSON object mapping each topic to a list of short facts, each ending with a `[source]` tag.
    - **Why:** You know these facts, so an invented fact in the article is easy to spot.
- `pipeline.py`
    - **What:** `steps_from_log(routing_log)` returns just the step names, and `run_pipeline(topic)` builds the graph, runs it with `start_state(topic)`, and returns a small dict: `article`, `accepted`, `reason`, `steps`.
    - **Why:** The API and the evals both call this one function. Without it, each would build and read the graph in its own way.
- `api/main.py`
    - **What:** Creates `app = FastAPI(...)`, calls `load_config()` at startup, includes the router, and adds `add_request_id`, a middleware that makes a short request ID, puts it on `request.state`, and returns it in the `X-Request-ID` header.
    - **Why:** uvicorn needs one app object to run. A missing key stops the server at startup, and every log line of a request can be found by its ID.
- `api/routes.py`

    - **What:** An `APIRouter` with three routes: `GET /health`; `POST /run-task`, which checks the rate limit, rejects a blank topic with 400, saves a "started" run, calls `run_pipeline()`, and on success saves the result — on a crash it marks the run failed, logs with `logger.exception`, and returns a plain 500; and `GET /runs/{run_id}`, which returns the run or a 404. Plus `check_rate_limit(caller_key)`, which raises 429.
    - **Why:** These are the addresses other programs call. Each failure gets the right HTTP status instead of a crash or a leaked traceback.
- `api/schemas.py`
    - **What:** Two Pydantic classes: `TaskRequest` (`topic: str` with a min and max length) and `TaskResponse` (`run_id`, `status`, `accepted`, `reason`).
    - **Why:** FastAPI checks every request against these shapes, so bad input gets a clear 422 instead of reaching the agents.
- `db/database.py`
    - **What:** `DB_PATH`, the `CREATE TABLE` SQL, and five functions: `get_connection()`, `insert_started_run(conn, topic)` (returns the new id from `lastrowid`), `finish_run()`, `fail_run()`, and `fetch_run(conn, run_id)` (returns a dict or `None`).
    - **Why:** Every run is saved with its status, so you can look it up later — even after a restart — and see which runs failed.
- `db/models.py`
    - **What:** One Pydantic class, `Run`: the fields of one saved run (`id`, `input`, `log`, `status`, `created_at`).
    - **Why:** The database code and the `GET /runs/{id}` response agree on the same fields.
- `evals/tasks.py`
    - **What:** Rule constants and a `TASKS` list of 10 dicts: each has an `id`, a `topic`, and either an `expected` text, `expect_no_article`, or judge `rules`.
    - **Why:** The same fixed test set every time, so two versions of the system can be compared fairly.
- `evals/judge.py`
    - **What:** A `JudgeResult` dataclass (`verdict`, `reason`), `call_model(prompt)`, `build_judge_prompt(task_input, output, rules)`, and `score_with_judge(...)`, which reads PASS or FAIL from the first line of the reply.
    - **Why:** Some qualities, like clarity, can't be checked with exact code. A judge model with written rules can score them.
- `evals/run_eval.py`
    - **What:** `run_one_task(task)` runs the pipeline and checks the article one of three ways (no article expected, an expected text, or the judge), and `run_eval_suite()` runs all tasks, prints the pass count and saves `report.json`.
    - **Why:** It gives one score for the whole system, so you know whether a change made it better or worse.
- `evals/regression_check.py`

    - **What:** `PASS_BAR`, a `SABOTAGE` text, `score(passed_by_id)`, `find_newly_failed(baseline, after)`, and `run_regression_check()`: run the suite, break one prompt on purpose, run it again, and report the drop.
    - **Why:** It proves the eval suite really catches a bad change. An eval that can never fail proves nothing.
- `Dockerfile`
    - **What:** Instructions to build the image: start `FROM python:3.11-slim`, copy and `pip install` the requirements, copy the code, `pip install -e .`, `EXPOSE 8000`, and start uvicorn with `CMD`.
    - **Why:** The API then runs the same way on any machine, and the key is only passed in when the container starts.
- `.dockerignore`
    - **What:** A list of paths to leave out of the image: `.env`, `.git`, `.venv/`, cache folders, `runs.db` and `report.json`.
    - **Why:** Your secret key and your local data never end up inside the image.
- `tests/api/test_routes.py`
    - **What:** pytest test functions that use `TestClient(app)` and `monkeypatch` to swap `routes.run_pipeline` for a fake, then `assert` the status code and body for each case (each step adds asserts or tests; names never change): health, a normal run that gets saved, a bad body (422), a blank topic (400), a crash (500), a missing run (404), the request ID header, and the rate limit (429).
    - **Why:** The API's behavior is tested quickly and for free, without any model calls.
- `tests/integration/test_judge.py`
    - **What:** Test functions that give the judge a known-good and a known-bad article and `assert` it passes the first and fails the second.
    - **Why:** You can only trust the judge's scores after checking it on examples where you already know the answer.

How imports work in this layout

As in Projects 1-4: an empty `__init__.py` in every folder with Python files (the new `api/`, `db/` and `evals/` too), `pip install -e .` once, imports like `from contentforge.db.database import fetch_run`, and every command run from the **repo root** — the prompts, `data/sample/facts.json` and `runs.db` are all found from there. uvicorn gets the app by its import path: `contentforge.api.main:app`. pytest finds the package because it is installed.

Left out on purpose (and when they come)

Folder / file in a big template	Why not here	Where it appears
Postgres, `docker-compose.yml`	One container and one SQLite file are enough	When several copies must share data (Doc12)
Redis for the rate limit	One process; the limit's weakness is written in the code	When you run several copies
A job queue for long runs	A run takes seconds, not minutes	Doc12's "Later" topics
Login / API keys for callers	A portfolio API, not a public product	Doc16 (security trade-offs)
CI/CD, a release gate, rollback	The suite runs by hand here	Doc19
`conftest.py`, fixtures	One small helper, `use_fake_team()`, does the job	When the test suite grows
Caching	Every request should get a fresh article	Doc12's caching topic
A frontend	`/docs` lets anyone try the API in the browser	Streamlit, when a project needs one

How the structure grows, step by step

Step	New files	Changed files
Setup	root files; Project 4's `src/`, `data/`, `tests/` copied in	—
1	`pipeline.py`, `api/` (`__init__.py`, `main.py`, `routes.py`, `schemas.py`), `tests/api/test_routes.py`	—
2	`db/` (`__init__.py`, `database.py`, `models.py`)	`api/schemas.py`, `api/routes.py`, `api/main.py`, `test_routes.py`
3	`Dockerfile`, `.dockerignore`	`api/routes.py`, `test_routes.py`
4	`evals/` (`__init__.py`, `tasks.py`, `judge.py`, `run_eval.py`, `regression_check.py`), `tests/integration/test_judge.py`	—

Setup (do this once, before Step 1)

Read first: Doc01 — Virtual environments, Doc01 — `.env` files. The same steps as Projects 1-4 — plus copying Project 4's code in.

1. Go into the repo folder, start git, and make a venv. From the curriculum's root folder (`multi-agent-ai-engineering-curriculum/`):

```
cd project_5_contentforge_pro_production
# the \ joins the two lines into one folder name
cd ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-\
Database-And-Docker
git init
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
```

2. Copy your finished Project 4 in — its `src/` , `data/` and `tests/` folders, unchanged:

```
# the \ joins the three lines into one path
P4=../../project_4_contentforge_multi_agent/\
ContentForge-Research-Write-Review-Team-Turns-Any-Topic-\
Into-A-Publish-Ready-Article
cp -r "$P4/src" "$P4/data" "$P4/tests" .
```

3. Create `.gitignore` **before** your first commit:

```
# secrets — never pushed
.env
# the virtual environment — rebuilt from requirements.txt
.venv/
# Python's compiled cache files
__pycache__/
*.pyc
# created by "pip install -e ."
*.egg-info/
# pytest's own cache folder
.pytest_cache/
# the run history (Step 2) — local data, not code
runs.db
# the eval report (Step 4) — made fresh on every run
report.json
```

4. Create `.env` and `.env.example` , the same as before:

```
# .env — never commit this file
OPENAI_API_KEY=sk-your-real-key-here
```

```
LOG_LEVEL=INFO
```

```
# .env.example — commit this one
# Copy this file to .env and fill in the real values. Never commit .env.
OPENAI_API_KEY=
LOG_LEVEL=INFO
```

5. Create `requirements.txt`:

```
fastapi>=0.110
langchain-core>=1.0
langchain-openai>=1.0
langgraph>=1.0
pydantic>=2.0
python-dotenv>=1.0
uvicorn[standard]>=0.29
# for the tests: TestClient runs on httpx
httpx>=0.27
pytest>=8.0
```

New since Project 4: `fastapi` (the API), `uvicorn` (the web server that runs it), `httpx` (what `TestClient` runs on) and `pytest`. SQLite needs nothing — `sqlite3` comes with Python. You'll also need Docker installed for Step 3 — a program on your computer, not a pip package.

6. Create `pyproject.toml` — Project 4's, with the new libraries and version `0.2.0`:

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "contentforge"
version = "0.2.0"
description = "ContentForge's 5-agent team, served as a production API."
requires-python = ">=3.10"
dependencies = [
    "fastapi>=0.110",
    "langchain-core>=1.0",
    "langchain-openai>=1.0",
    "langgraph>=1.0",
    "pydantic>=2.0",
    "python-dotenv>=1.0",
    "uvicorn[standard]>=0.29",
]
```

```
[project.scripts]
# after "pip install -e .", typing "contentforge" runs main()
contentforge = "contentforge.main:main"

[tool.setuptools.packages.find]
# the package code lives under src/, not in the repo root
where = ["src"]
```

7. Install, and check that Project 4 still runs in its new home:

```
pip install -r requirements.txt
pip install -e .
python -m contentforge.main           # Project 4's terminal app
```

Check it worked: `python -c "import contentforge; print('ok')"` prints `ok`, and `python -m contentforge.main` writes an article, exactly as in Project 4.

8. Create `LICENSE` — the MIT license, so others may reuse your code. Put the current year and your own name on the Copyright line:

```
MIT License

Copyright (c) 2026 Your Name

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

Why This Project Is Good for Your Portfolio

- **What problem it solves:** turns Project 4 from "works when I run it" into a service someone else can call, trust with real traffic, and debug afterwards.
- **Why it matters:** this is the project you link, not just describe — an API with a clear contract, a saved history, a clean Dockerfile, and tests that would catch a real regression.
- **When you'd build something like this at a job:** this *is* the everyday shape of production AI work — wrap an agent system in a service, save its history, test it, and defend each trade-off.
- **How it's built:** FastAPI routes on an `APIRouter`, Pydantic shapes, SQLite with a 3-state run status, request-ID logs, a per-caller rolling-window rate limit, Docker with secrets only at run time, pytest with a faked team, and an LLM-judge eval suite with a regression check.

Problems you'll likely run into, and how to fix them:

Problem	Fix
`MissingConfigError` when the tests start	The app refuses to start without `OPENAI_API_KEY` — keep it in `.env` (the API tests fake the team, so any value works)
An API test calls the real model	Swap `routes.run_pipeline`, the name the route uses — not `pipeline.run_pipeline`
Tests get a surprise `429`	Reset `routes.request_times` in every test (`use_fake_team()` does it)
A client sees a traceback or `str(exc)`	Catch at the route, `logger.exception()` on the server, a plain 500 to the client
A crashed run leaves no record	Save the `"started"` row in its own `with conn:` **before** the team runs; mark it `"failed"` on a crash
`curl` gets "connection refused" from the container	`--host 0.0.0.0` in `CMD`, and `-p 8000:8000` on `docker run`
The key ends up in the image	`.env` in `.dockerignore`; pass it with `--env-file` at run time; check with `docker history`
The eval score doesn't drop after the sabotage	A rule is too loose, or another agent repaired the damage — tighten the rule, or break something no later agent can fix

Built During These Documents

12_production_engineering → 13_testing_evaluation_observability (on top of Docs 01-11 and Project 4)

Plan Before You Code

See 15_five_projects_index: write down Problem → Requirements → Architecture → Components → Data Flow → Implementation Plan → Coding Tasks — and draw one request's path: middleware → rate limit → database → team → database → reply.

How To Build This — Step by Step

The project grows step by step, and **after every step it runs**: Step 1 answers over HTTP, Step 2 saves every run, Step 3 runs in a container behind a rate limit, Step 4 measures the articles themselves. Build it yourself first; use the hints if you're stuck, and the solution only to compare.

Every command runs from the **repo root** with the venv active.

Step 1 — ContentForge, Reachable Over HTTP for the First Time

Project: ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-Database-And-Docker — Step 1 of 4

- **Where you are now:** Setup is done — Project 4 runs in the new repo, but only from the terminal.
- **What's new in this step:** `pipeline.py` (the one door into the team), the `api/` package (`schemas.py`, `routes.py`, `main.py`), and `tests/api/test_routes.py` with the team faked.
- **What stays the same:** every Project 4 file.
- **Why:** everything later builds on the HTTP layer. Prove it alone — no database, no Docker — so a later bug can't hide in it.
- **How:** `POST /run-task` → our own 400 for a blank topic → `run_pipeline(topic)` → a checked `TaskResponse`; any crash → `logger.exception()` + a plain 500. Tests: `TestClient` + `monkeypatch`.
- **Working after this step:** `uvicorn contentforge.api.main:app --reload` answers `POST /run-task`; `pytest tests/api` passes with no model calls.

Read first: Doc12 — health route, Doc12 — chat route, Doc13 — testing layers, Doc13 — test a tool without the LLM (`monkeypatch`).

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Write `pipeline.py`: `run_pipeline(topic)` runs the graph and returns `article`, `accepted`, `reason` and `steps`.
2. Write `api/schemas.py`: `TaskRequest(topic)` with `Field(min_length=1, max_length=200)`, and `TaskResponse(accepted, reason, article)`.
3. Write `api/routes.py`: an `APIRouter` with `GET /health` and `POST /run-task` (400 for a blank topic, a plain 500 for a crash).

4. Write `api/main.py`: `load_config()` first, then the app and `include_router(router)`.
5. Write `tests/api/test_routes.py`: one pytest test per case, with `monkeypatch` swapping `routes.run_pipeline` for a fake.
6. Run `pytest tests/api -v`, then start uvicorn and send one **real** topic from `/docs`.

The root files — `.gitignore`, `.env`, `.env.example`, `requirements.txt`, `pyproject.toml` and `LICENSE` — come from Setup; this step adds the files below.

Your files after Step 1:

```
src/contentforge/
├── pipeline.py         (new) run_pipeline(): the one door in
└── api/
    ├── __init__.py     (new)
    ├── main.py         (new) load_config(), the app
    ├── routes.py       (new) /health, /run-task
    └── schemas.py      (new) TaskRequest, TaskResponse
tests/api/
└── test_routes.py      (new) 5 tests, team faked
```

Why these files (what and why):

- `pipeline.py`
  - **What:** `steps_from_log(routing_log)` returns just the step names, and `run_pipeline(topic)` builds the graph, runs it with `start_state(topic)`, and returns a small dict: `article`, `accepted`, `reason`, `steps`.
  - **Why:** The API and the evals both call this one function. Without it, each would build and read the graph in its own way.
- `main.py`
  - **What:** `main()` loads the config, builds the graph, reads a topic, runs `graph.invoke(start_state(topic))`, and passes the result to `print_report(result)`. `print_report()` (written in Step 2, extended in Step 5) prints the routing log and then the article, the "couldn't agree" report, or "nothing is known".
  - **Why:** One starting point that reads the topic and prints the result. The team and its routing live in other files.
- `api/routes.py`
  - **What:** An `APIRouter` with three routes: `GET /health`; `POST /run-task`, which checks the rate limit, rejects a blank topic with 400, saves a "started" run, calls `run_pipeline()`, and on success saves the result — on a crash it marks the run failed, logs with `logger.exception`, and returns a plain 500; and `GET /runs/{run_id}`, which returns the run or a 404. Plus `check_rate_limit(caller_key)`, which raises 429.
  - **Why:** These are the addresses other programs call. Each failure gets the right HTTP status instead of a crash or a leaked traceback.

- `schemas.py`
  - **What:** One Pydantic class, `ReviewVerdict`, with `score: int = Field(ge=1, le=10)` and `feedback: str`.
  - **Why:** The Reviewer's score is a checked number the code compares with the pass mark — not a word to search for.
- `test_routes.py`
  - **What:** pytest test functions that use `TestClient(app)` and `monkeypatch` to swap `routes.run_pipeline` for a fake, then `assert` the status code and body for each case (each step adds asserts or tests; names never change): health, a normal run that gets saved, a bad body (422), a blank topic (400), a crash (500), a missing run (404), the request ID header, and the rate limit (429).
  - **Why:** The API's behavior is tested quickly and for free, without any model calls.

Step 2 — Every Run Saved to a Database and Findable by ID

Project: ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-Database-And-Docker — Step 2 of 4

- **Where you are now:** after Step 1 the API answers, but a run's result exists only in the reply — once sent, it's gone.
- **What's new in this step:** the `db/` package (`database.py`, `models.py`), `GET /runs/{run_id}`, a `run_id` and `status` in every reply, and a request ID on every log line.
- **What stays the same:** the team, `pipeline.py`, the 400/500 rules, the request shape.
- **Why:** "what happened on that run yesterday?" needs a real answer — and a crashed run must leave an honest record, not vanish.
- **How:** `insert_started_run()` **before** the team runs → `finish_run()` or `fail_run()` after, each in its own `with conn:`; a middleware makes the request ID; `fetch_run()` + 404.
- **Working after this step:** every `POST /run-task` saves a row; `GET /runs/1` shows its input, article, routing log and status.

Read first: Doc12 — SQLite persistence, Doc12 — validation 422, Doc12 — chat route (Approach 4, the request ID).

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Write `db/database.py`: `DB_PATH`, `get_connection()`, `insert_started_run()`, `finish_run()`, `fail_run()`, `fetch_run()`.
2. Write `db/models.py`: `Run`, the shape `GET /runs/{run_id}` returns.
3. Add `run_id` and `status` to `TaskResponse`.
4. Change `routes.py`: save `"started"` first, `"completed"` or `"failed"` after; the routing steps go in the `log` column; add `GET /runs/{run_id}` with a 404.

5. Add the request-ID middleware to `api/main.py`, and put `[request_id]` at the start of every log line.
6. Add tests: the row is saved, a 422 saves nothing, a crash is saved as `"failed"`, a wrong ID is a 404, every reply has `X-Request-ID`.

Your files after Step 2:

```
src/contentforge/
├── api/
│   ├── main.py         (changed) + request-ID middleware
│   ├── routes.py       (changed) saves runs, + /runs/{run_id}
│   └── schemas.py      (changed) + run_id, status
└── db/
    ├── __init__.py     (new)
    ├── database.py     (new) insert/finish/fail/fetch a run
    └── models.py       (new) Run
tests/api/test_routes.py (changed) + saved-row checks, 404
```

Why these files (what and why):

- `main.py`
  - **What:** `main()` loads the config, builds the graph, reads a topic, runs `graph.invoke(start_state(topic))`, and passes the result to `print_report(result)`. `print_report()` (written in Step 2, extended in Step 5) prints the routing log and then the article, the "couldn't agree" report, or "nothing is known".
  - **Why:** One starting point that reads the topic and prints the result. The team and its routing live in other files.
- `api/routes.py`
  - **What:** An `APIRouter` with three routes: `GET /health`; `POST /run-task`, which checks the rate limit, rejects a blank topic with 400, saves a "started" run, calls `run_pipeline()`, and on success saves the result — on a crash it marks the run failed, logs with `logger.exception`, and returns a plain 500; and `GET /runs/{run_id}`, which returns the run or a 404. Plus `check_rate_limit(caller_key)`, which raises 429.
  - **Why:** These are the addresses other programs call. Each failure gets the right HTTP status instead of a crash or a leaked traceback.
- `schemas.py`
  - **What:** One Pydantic class, `ReviewVerdict`, with `score: int = Field(ge=1, le=10)` and `feedback: str`.
  - **Why:** The Reviewer's score is a checked number the code compares with the pass mark — not a word to search for.
- `db/database.py`

    - **What:** `DB_PATH`, the `CREATE TABLE` SQL, and five functions: `get_connection()`, `insert_started_run(conn, topic)` (returns the new id from `lastrowid`), `finish_run()`, `fail_run()`, and `fetch_run(conn, run_id)` (returns a dict or `None`).
    - **Why:** Every run is saved with its status, so you can look it up later — even after a restart — and see which runs failed.
- `db/models.py`
    - **What:** One Pydantic class, `Run`: the fields of one saved run (`id`, `input`, `log`, `status`, `created_at`).
    - **Why:** The database code and the `GET /runs/{id}` response agree on the same fields.
- `tests/api/test_routes.py`
    - **What:** pytest test functions that use `TestClient(app)` and `monkeypatch` to swap `routes.run_pipeline` for a fake, then `assert` the status code and body for each case (each step adds asserts or tests; names never change): health, a normal run that gets saved, a bad body (422), a blank topic (400), a crash (500), a missing run (404), the request ID header, and the rate limit (429).
    - **Why:** The API's behavior is tested quickly and for free, without any model calls.

Step 3 — Containerized and Rate-Limited: Runs the Same Everywhere, Survives Abuse

Project: ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-Database-And-Docker — Step 3 of 4

- **Where you are now:** after Step 2 the API saves every run — but it runs only on your laptop, and anyone can call it as often as they like.
- **What's new in this step:** a per-caller rolling-window rate limit (`429`), a `Dockerfile`, a `.dockerignore`, and a test for the 429.
- **What stays the same:** the routes' other rules, the database, the team.
- **Why:** each run costs five agents' worth of model calls — one script in a loop could empty your budget. And "it works on my machine" isn't something you can hand to anyone.
- **How:** `check_rate_limit(http_request.client.host)` first in `POST /run-task`; the image installs the requirements, then the package, and starts uvicorn on `0.0.0.0`; the key arrives only with `docker run --env-file .env`.
- **Working after this step:** the API runs from a container; the sixth quick request from one caller gets a 429; `docker run` without a key stops at once with a clear error.

Read first: Doc12 — dead connection and container, Doc12 Build Task (the rate limiter).

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Add the rate limiter to `routes.py`: `RATE_LIMIT`, `RATE_WINDOW_SECONDS`, `request_times`, `check_rate_limit(caller_key)` — and write its limitation in a comment.

2. Call it first in `POST /run-task`, with `http_request.client.host`.
3. In the tests, reset `request_times` in `use_fake_team()`, and add a 429 test.
4. Write the `Dockerfile` and `.dockerignore`.
5. `docker build`, `docker run` with `--env-file .env`, and `curl localhost:8000/health`. Check `docker history` has no key, and that `docker run` **without** the key fails loudly.

Your files after Step 3:

```
src/contentforge/api/
└── routes.py           (changed) + check_rate_limit(), 429
tests/api/test_routes.py (changed) + the 429 test
Dockerfile              (new) the API's image
.dockerignore           (new) no .env, no runs.db in the image
```

Why these files (what and why):

- `api/routes.py`
  - **What:** An `APIRouter` with three routes: `GET /health`; `POST /run-task`, which checks the rate limit, rejects a blank topic with 400, saves a "started" run, calls `run_pipeline()`, and on success saves the result — on a crash it marks the run failed, logs with `logger.exception`, and returns a plain 500; and `GET /runs/{run_id}`, which returns the run or a 404. Plus `check_rate_limit(caller_key)`, which raises 429.
  - **Why:** These are the addresses other programs call. Each failure gets the right HTTP status instead of a crash or a leaked traceback.
- `tests/api/test_routes.py`
  - **What:** pytest test functions that use `TestClient(app)` and `monkeypatch` to swap `routes.run_pipeline` for a fake, then `assert` the status code and body for each case (each step adds asserts or tests; names never change): health, a normal run that gets saved, a bad body (422), a blank topic (400), a crash (500), a missing run (404), the request ID header, and the rate limit (429).
  - **Why:** The API's behavior is tested quickly and for free, without any model calls.
- `Dockerfile`
  - **What:** Instructions to build the image: start `FROM python:3.11-slim`, copy and `pip install` the requirements, copy the code, `pip install -e .`, `EXPOSE 8000`, and start uvicorn with `CMD`.
  - **Why:** The API then runs the same way on any machine, and the key is only passed in when the container starts.
- `.dockerignore`
  - **What:** A list of paths to leave out of the image: `.env`, `.git`, `.venv/`, cache folders, `runs.db` and `report.json`.
  - **Why:** Your secret key and your local data never end up inside the image.

Step 4 — A Test Suite That Catches a Bad Change Before It Ships (final)

Project: ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-Database-And-Docker — Step 4 of 4

- **Where you are now:** after Step 3 the service is solid — but its tests fake the team, so they can't see a **worse article**.
- **What's new in this step:** the `evals/` package (`tasks.py`, `judge.py`, `run_eval.py`, `regression_check.py`) and `tests/integration/test_judge.py`. Project 4's tests stay exactly as they are.
- **What stays the same:** the API, the database, Docker, the team.
- **Why:** a prompt edit that quietly drops the `[source]` tags would pass every API test. Only a suite that scores the real output — and has been seen to fail — catches it.
- **How:** 10 fixed topics with checks written first → `run_pipeline()` → a free code check, or the judge for meaning → a score and `report.json`; then break Research's prompt on purpose and prove the score falls below the bar.
- **Working after this step:** the full project — the unit and API tests pass for free, `run_eval` scores 10/10, and `regression_check` reports the drop.

Read first: Doc13 — a normal, exact test, Doc13 — fix a flaky test, Doc13 — score an answer, Doc13 — regression catch, Doc13 Build Task.

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Leave Project 4's tests (`tests/unit/*`, `tests/integration/test_live_agents.py`) exactly as they are — this step only adds tests.
2. Write `evals/tasks.py` — 10 topics: exact `[source]` tags, judge rules, and unknown topics that must give no article. **Write them before running anything.**
3. Write `evals/judge.py` (Doc13's judge) and `evals/run_eval.py` (calls `run_pipeline()`, prints the score, saves `report.json`).
4. Write `tests/integration/test_judge.py`: the judge must PASS a good article and FAIL two bad ones.
5. Write `evals/regression_check.py`, run it, and delete the words it asks for from `research.txt` when it pauses. Put them back afterwards.

Your files after Step 4 (final): the full tree in Project Structure. What changed:

```
src/contentforge/evals/
├── __init__.py          (new)
├── tasks.py             (new) 10 topics, checks written first
├── judge.py             (new) score_with_judge()
├── run_eval.py          (new) the score, report.json
```

```
└── regression_check.py  (new) sabotage, prove the drop
tests/integration/
└── test_judge.py        (new) calibration set
```

Why these files (what and why):

- `evals/tasks.py`
  - **What:** Rule constants and a `TASKS` list of 10 dicts: each has an `id`, a `topic`, and either an `expected` text, `expect_no_article`, or judge `rules`.
  - **Why:** The same fixed test set every time, so two versions of the system can be compared fairly.
- `evals/judge.py`
  - **What:** A `JudgeResult` dataclass (`verdict`, `reason`), `call_model(prompt)`, `build_judge_prompt(task_input, output, rules)`, and `score_with_judge(...)`, which reads PASS or FAIL from the first line of the reply.
  - **Why:** Some qualities, like clarity, can't be checked with exact code. A judge model with written rules can score them.
- `evals/run_eval.py`
  - **What:** `run_one_task(task)` runs the pipeline and checks the article one of three ways (no article expected, an expected text, or the judge), and `run_eval_suite()` runs all tasks, prints the pass count and saves `report.json`.
  - **Why:** It gives one score for the whole system, so you know whether a change made it better or worse.
- `evals/regression_check.py`
  - **What:** `PASS_BAR`, a `SABOTAGE` text, `score(passed_by_id)`, `find_newly_failed(baseline, after)`, and `run_regression_check()`: run the suite, break one prompt on purpose, run it again, and report the drop.
  - **Why:** It proves the eval suite really catches a bad change. An eval that can never fail proves nothing.
- `test_judge.py`
  - **What:** Test functions that give the judge a known-good and a known-bad article and `assert` it passes the first and fails the second.
  - **Why:** You can only trust the judge's scores after checking it on examples where you already know the answer.

Final Deliverable: ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-Database-And-Docker — Project 4's five agents behind a FastAPI service: typed requests, safe errors, every run saved and findable by ID, request-ID logs, a per-caller rate limit, a Docker image with no secrets inside, API tests with the team faked, and an eval suite proven to catch a bad prompt change.

Run, Test and Configure the Final Project

The same commands and settings as the repo's README.md — once Step 4 is done, this is how the finished project is run, tested and configured.

Configuration

Variable	Required	Default	Meaning
`OPENAI_API_KEY`	yes	—	Your OpenAI API key; the app will not start without it
`LOG_LEVEL`	no	`INFO`	`DEBUG`, `INFO`, `WARNING` or `ERROR`

Setting	Where	Default
Chat model	`models/llm.py` → `MODEL_NAME`	`gpt-4o-mini`
Pass mark	`agents/reviewer_agent.py` → `PASS_SCORE`	`7` (out of 10)
Revision limit	`agents/supervisor.py` → `MAX_REVISIONS`	`3`
Topic length	`api/schemas.py` → `TaskRequest`	1-200 characters
Rate limit	`api/routes.py` → `RATE_LIMIT`, `RATE_WINDOW_SECONDS`	5 requests per 60 s, per caller
Database file	`db/database.py` → `DB_PATH`	`runs.db`
Eval pass bar	`evals/regression_check.py` → `PASS_BAR`	`0.8`
Known facts	`data/sample/facts.json`	electric bikes, remote work, solar panels

Usage

Run from the repo root:

```
uvicorn contentforge.api.main:app --reload     # the API on port 8000
python -m contentforge.main                    # the terminal app
```

Open `http://localhost:8000/docs` to try every route in the browser.

Route	What it does
`GET /health`	`{"status":"ok"}`

Route	What it does
`POST /run-task`	Body `{"topic": "..."}`; returns `run_id`, `status`, `accepted`, `reason`, `article`
`GET /runs/{run_id}`	The saved run: `input`, `output`, `log`, `status`, `created_at`; 404 if unknown

In a container:

```
docker build -t contentforge-pro .
docker run -p 8000:8000 --env-file .env contentforge-pro
curl localhost:8000/health
```

Running the Tests

Run from the repo root:

```
# free — no model calls
python tests/unit/test_lookup.py
python tests/unit/test_supervisor.py
pytest tests/api -v

# real API calls — needs a valid OPENAI_API_KEY
python tests/integration/test_live_agents.py
pytest tests/integration/test_judge.py -v
```

The unit tests (plain scripts, the same as in the original team project) check the lookup tool and every routing rule. The API tests check every route and status code — 200, 400, 404, 422, 429 and a plain 500 — and the saved rows, with the team replaced by a fake. The integration tests run the real team and check the judge on answers with a known verdict.

Article quality (real API calls):

```
python -m contentforge.evals.run_eval           # score + report.json
python -m contentforge.evals.regression_check   # prove a bad change drops it
```

`run_eval` runs 10 fixed topics — exact `[source]` checks, judge rules and unknown topics — and prints the score. `regression_check` runs it, pauses while you remove the `[source]` rule from Research's prompt, runs it again, and reports whether the score fell below the bar.

Checklist Before You Call This Done

- [ ] The repo folder name matches the GitHub repo name exactly
- [ ] Project 4's files are unchanged; the API calls the team only through `run_pipeline()`
- [ ] `POST /run-task` has typed shapes: 422 for a bad body, 400 for a blank topic, a plain 500 for a crash (the traceback only in the server log)
- [ ] Every run is saved (`"started"` → `"completed"` / `"failed"`) and found with `GET /runs/{run_id}`; a wrong ID is a 404
- [ ] Every log line for a request starts with the same request ID, also sent back as `X-Request-ID`
- [ ] A per-caller rate limit returns 429, and its limitation is written in the code
- [ ] `docker run` works with the key passed at run time; `docker history` shows no key; no key → a clear startup error
- [ ] The unit tests and `pytest tests/api` pass with no model calls
- [ ] The eval suite scores the real articles, and the regression check proves a sabotaged prompt drops the score
- [ ] The judge passes its own calibration test
- [ ] `.env`, `runs.db` and `report.json` are not in git
- [ ] You can defend the trade-offs: SQLite vs. Postgres, the in-memory rate limit, the synchronous route

Publish It to GitHub

1. Copy README.md from this folder into your repo folder, and put your own name in its License line.
2. Check nothing secret is staged, then commit:

```
git status                  # .env, .venv/, runs.db must NOT appear
git add .
git commit -m "ContentForge Pro: API, run history, Docker, test suite"
```

3. On GitHub, create a new repo named exactly `ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-Database-And-Docker` (no README — you have one). In the repo's **Description** box, paste the **Short description** line from the top of README.md. Then:

```
git branch -M main
git remote add origin <your-repo-url>
git push -u origin main
```

Status

Not started. Track your progress in ../PROGRESS.md.

Stuck? Ask for **Hint 1** or **Hint 2** about the exact part you're stuck on. Only ask for the full code if you say **"Show me the solution."**